MethodTabPanel - Method切り替えタブパネル

1. 概要

1.1 コンポーネントの目的

MethodTabPanelは、複数のMethod（実験方法）を切り替えるタブUIコンポーネントです。各Methodの表示、追加、編集、削除機能を提供し、Method間の切り替えを容易にします。

主要機能:

- Method切り替えタブ表示
- ノード数表示（各Methodごと）
- Method追加
- Method ID編集
- Method削除（条件付き）
- アクティブMethod強調表示

1.2 使用場面

- PetriNetEditorInner内で常に表示
- EditorToolbarの下、React Flowキャンバスの上に配置
- 複数Methodを管理する際の切り替えインターフェース

1.3 アクセス権限

- **必須ロール**: Experiment Admin
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

```
┌──────────────────────────────────────────────────────────────┐
│ [Method1] (3 nodes) [編集] [削除] │ [Method2] (0 nodes) [編集] │
│ [削除] │ [+]                                                 │
└──────────────────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 Methodタブ

- **表示要素**:
  - Method ID（ラベル）
  - ノード数Tag（"X nodes"）
    - アクティブタブ: 青色Tag
    - 非アクティブタブ: デフォルト色Tag
  - 編集ボタン（EditOutlinedアイコン）
  - 削除ボタン（CloseOutlinedアイコン、条件付き表示）

2.2.2 Method追加ボタン

- **位置**: タブリストの右端
- **アイコン**: PlusOutlined
- **ツールチップ**: "Add new method"
- **スタイル**: type="text"

2.2.3 Method ID編集モーダル

- **タイトル**: "Method ID編集"
- **入力フィールド**: Method IDテキスト入力
- **バリデーション**:
  - 必須チェック

    - ユニーク性チェック

- **ボタン**:
    - 保存ボタン
    - キャンセルボタン

2.2.4 Method削除確認モーダル

- **タイトル**: "Method削除"
- **内容**: `Method "XXX" を削除しますか？`
- **ボタン**:
    - 削除ボタン（danger）
    - キャンセルボタン

---

3. 機能仕様

3.1 Props定義

```
export interface MethodTabPanelProps {
  // methodデータ配列
  methods: MethodData[];

  // 現在アクティブなmethodのindex
  activeMethodIndex: number;

  // 現在編集中のノード配列（リアルタイム更新用）
  currentNodes?: Node[];

  // タブ切り替え時のコールバック
  onChange?: (index: number) => void;

  // Method追加時のコールバック
  onAddMethod?: () => void;

  // MethodID編集時のコールバック
  onEditMethodId?: (index: number, newMethodId: string) => void;
```

```
  // Method削除時のコールバック
  onDeleteMethod?: (index: number) => void;
}
```

Props説明:

Prop名	型	必須	説明
methods	MethodData[]	✅	Methodデータ配列
activeMethodIndex	number	✅	アクティブなMethodのインデックス
currentNodes	Node[]	-	現在編集中のノード配列（リアルタイム更新用）
onChange	(index) => void	-	タブ切り替えコールバック
onAddMethod	() => void	-	Method追加コールバック
onEditMethodId	(index, newId) => void	-	Method ID編集コールバック
onDeleteMethod	(index) => void	-	Method削除コールバック

MethodData型:

```
interface MethodData {
  methodId: string;
  name?: string;
  nodes?: Node[];
  edges?: Edge[];
}
```

3.2 State定義

```
const [editDialogOpen, setEditDialogOpen] = useState(false);
const [editingMethodIndex, setEditingMethodIndex] = useState<number | null>(null);
const [newMethodId, setNewMethodId] = useState('');
const [idError, setIdError] = useState('');
```

State説明:

State名	型	初期値	説明

editDialogOpen	boolean	false	Method ID編集モーダルの開閉状態
editingMethodIndex	number | null	null	編集中のMethodインデックス
newMethodId	string	''	新しいMethod ID（編集用）
idError	string	''	バリデーションエラーメッセージ

3.3 タブ切り替え

3.3.1 処理フロー

```
const handleChange = (activeKey: string) => {
  const index = parseInt(activeKey, 10);
  if (onChange) {
    onChange(index);
  }
};
```

3.3.2 UI動作

- ユーザー操作: Methodタブをクリック
- 実行: 親コンポーネント（PetriNetEditorInner）のhandleMethodChange実行
- 結果:
    1. activeMethodIndexが更新される
    2. React Flowキャンバスに選択したMethodのノード・エッジが表示される
    3. アクティブタブのTagが青色に変化

3.4 Method追加

3.4.1 処理フロー

```
const handleAddMethod = () => {
  if (onAddMethod) {
    onAddMethod();
```

```
  }
};
```

3.4.2 UI動作

- ユーザー操作: [+]ボタンをクリック
- 実行: 親コンポーネントのhandleAddMethod実行
- 結果:
  1. 新しいMethodが追加される（methodId: `method_${Date.now()}`）
  2. 新しいタブが表示される
  3. 新しいMethodがアクティブになる

3.5 Method ID編集

3.5.1 処理フロー

```
// 編集ダイアログを開く
const handleOpenEditDialog = (index: number, event: React.MouseEvent) => {
  event.stopPropagation(); // タブ切り替えを防止
  setEditingMethodIndex(index);
  setNewMethodId(methods[index].methodId);
  setIdError('');
  setEditDialogOpen(true);
};

// Method ID保存
const handleSaveMethodId = () => {
  // バリデーション
  if (!newMethodId || newMethodId.trim() === '') {
    setIdError('Method IDは必須です');
    return;
  }

  // ユニーク性チェック
  const isDuplicate = methods.some((m, idx) =>
    idx !== editingMethodIndex && m.methodId === newMethodId
  );

  if (isDuplicate) {
```

```
    setIdError('このMethod IDは既に使用されています');
    return;
  }

  // 保存
  if (onEditMethodId && editingMethodIndex !== null) {
    onEditMethodId(editingMethodIndex, newMethodId);
  }

  handleCloseEditDialog();
};
```

3.5.2 UI動作

- ユーザー操作: [編集]ボタンをクリック
- モーダル表示: "Method ID編集"
- 入力: 新しいMethod IDを入力
- バリデーション:
  - 必須チェック: 空の場合エラー
  - ユニーク性チェック: 重複する場合エラー
- 保存: 親コンポーネントのhandleEditMethodId実行
- 結果: Method IDが更新され、タブラベルが変更される

3.6 Method削除

3.6.1 処理フロー

```
const handleDeleteMethod = (index: number, event: React.MouseEvent) => {
  event.stopPropagation(); // タブ切り替えを防止

  const method = methods[index];

  // アクティブなタブの場合は現在のノード数を使用、それ以外はmethod.nodesを使用
  const nodeCount = index === activeMethodIndex
    ? (currentNodes?.length || 0)
    : (method.nodes?.length || 0);

  // 0 nodeの場合のみ削除可能
```

```
  if (nodeCount > 0) {
    message.warning(`このMethodには${nodeCount}個のノードが含まれています。削除するには、まずすべてのノードを削除してください。`);
    return;
  }

  // 最後の1つは削除不可
  if (methods.length === 1) {
    message.warning('最後のMethodは削除できません。');
    return;
  }

  // 確認ダイアログ
  Modal.confirm({
    title: 'Method削除',
    content: `Method "${method.methodId}" を削除しますか？`,
    okText: '削除',
    cancelText: 'キャンセル',
    okType: 'danger',
    onOk: () => {
      if (onDeleteMethod) {
        onDeleteMethod(index);
      }
    },
  });
};
```

3.6.2 UI動作

- ユーザー操作: [削除]ボタンをクリック
- バリデーション:
  1. ノード数チェック: ノードが1個以上あればwarning表示、削除中止
  2. 最後のMethodチェック: methods.length === 1の場合warning表示、削除中止
- 確認モーダル: "Method削除"
- 削除実行: 親コンポーネントのhandleDeleteMethod実行
- 結果:
  1. Methodが削除される
  2. アクティブMethodが調整される（削除したMethodがアクティブだった場合）
  3. タブが1つ減る

3.7 ノード数表示

3.7.1 処理フロー

```
// アクティブなタブは現在のノード数を使用、それ以外はmethod.nodesを使用
const nodeCount = index === activeMethodIndex
  ? (currentNodes?.length || 0)
  : (method.nodes?.length || 0);
```

3.7.2 UI動作

- **アクティブタブ**: currentNodes?.lengthを使用（リアルタイム更新）
- **非アクティブタブ**: method.nodes?.lengthを使用（保存済みデータ）
- **表示形式**: "X nodes"（Tagで表示）
- **色**:
  - アクティブタブ: 青色
  - 非アクティブタブ: デフォルト色

---

4. 削除ボタン表示条件

4.1 表示条件

削除ボタンは以下の条件をすべて満たす場合のみ表示されます：

```
{nodeCount === 0 && methods.length > 1 && (
  <Button
    type="text"
    size="small"
    danger
    icon={<CloseOutlined style={{ fontSize: 12 }} />}
    onClick={(e) => handleDeleteMethod(index, e)}
```

```
  />
)}
```

条件:

1. `nodeCount === 0` : ノードが0個
2. `methods.length > 1` : Methodが2個以上（最後の1つは削除不可）

理由:

- ノードがあるMethodは削除できない（データ保護）
- 最後のMethodは削除できない（最低1つのMethodが必要）

---

5. スタイル仕様

5.1 コンテナスタイル

```
<div style={{ width: '100%', display: 'flex', alignItems: 'center' }}>
  <Tabs
    type="card"
    size="small"
    style={{ flex: 1, margin: 0 }}
  />

  <div style={{
    padding: '0 8px',
    borderBottom: '1px solid #f0f0f0',
    backgroundColor: '#fff',
  }}>
    {/* Add Method Button */}
  </div>
</div>
```

説明:

- Flexboxレイアウト
- Tabsが伸縮（flex: 1）

- 追加ボタンは固定幅

5.2 タブラベルスタイル

```
<Space size={4}>
  <span>{method.methodId}</span>
  <Tag
    color={index === activeMethodIndex ? 'blue' : 'default'}
    style={{ fontSize: 11, lineHeight: '16px' }}
  >
    {nodeCount} nodes
  </Tag>
  <Button style={{ padding: '0 4px', height: 20 }} />
</Space>
```

5.3 methodsが空の場合

```
<div style={{
  width: '100%',
  display: 'flex',
  alignItems: 'center',
  justifyContent: 'center',
  padding: '8px 0',
  borderBottom: '1px solid #f0f0f0',
  backgroundColor: '#fff',
}}>
  <Tooltip title="Add new method">
    <Button type="text" icon={<PlusOutlined />} />
  </Tooltip>
</div>
```

---

6. エラーハンドリング

6.1 エラーケース

エラーケース	対処方法
Method IDが空	idError設定、保存中止
Method IDが重複	idError設定、保存中止
ノードがあるMethodを削除	message.warning表示、削除中止
最後のMethodを削除	message.warning表示、削除中止

6.2 バリデーション

- Method ID編集時: 必須チェック + ユニーク性チェック
- Method削除時: ノード数チェック + 最後のMethodチェック

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/MethodTabPanel.tsx`

7.2 関連ファイル

- **型定義**: `frontend/src/types/editor.ts` (MethodData)

7.3 依存コンポーネント

- Ant Design:
  - Tabs
  - Tag
  - Button
  - Tooltip
  - Modal
  - Input

    - Space
    - message
- Ant Design Icons:
    - PlusOutlined
    - EditOutlined
    - CloseOutlined
- React Flow:
    - Node

7.4 親コンポーネント

- PetriNetEditorInner

---

8. 使用例

8.1 基本的な使用

```
<MethodTabPanel
  methods={methods}
  activeMethodIndex={activeMethodIndex}
  currentNodes={nodes}
  onChange={handleMethodChange}
  onAddMethod={handleAddMethod}
  onEditMethodId={handleEditMethodId}
  onDeleteMethod={handleDeleteMethod}
/>
```

8.2 親コンポーネントでの実装例

```
// Method切り替え
const handleMethodChange = (index: number) => {
  setActiveMethodIndex(index);

  // 選択したMethodのノード・エッジを読み込み
  const method = methods[index];
  setNodes(method.nodes || []);
  setEdges(method.edges || []);
};

// Method追加
const handleAddMethod = () => {
  const newMethod: MethodData = {
    methodId: `method_${Date.now()}`,
    name: '',
    nodes: [],
    edges: [],
  };

  setMethods([...methods, newMethod]);
  setActiveMethodIndex(methods.length); // 新しいMethodをアクティブに
};

// Method ID編集
const handleEditMethodId = (index: number, newMethodId: string) => {
  const updatedMethods = [...methods];
  updatedMethods[index] = {
    ...updatedMethods[index],
    methodId: newMethodId,
  };
  setMethods(updatedMethods);
};

// Method削除
const handleDeleteMethod = (index: number) => {
  const updatedMethods = methods.filter((_, i) => i !== index);
  setMethods(updatedMethods);

  // アクティブMethodの調整
  if (index === activeMethodIndex) {
    setActiveMethodIndex(Math.max(0, index - 1));
  } else if (index < activeMethodIndex) {
    setActiveMethodIndex(activeMethodIndex - 1);
  }
};
```

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を新規作成	Cline
2026-01-29	Material-UI → Ant Design変換	Development Team